Aula 05 – Caminho de dados e controle MIPS Monociclo

Prof. João Fernando Mari
joaofmari.github.io

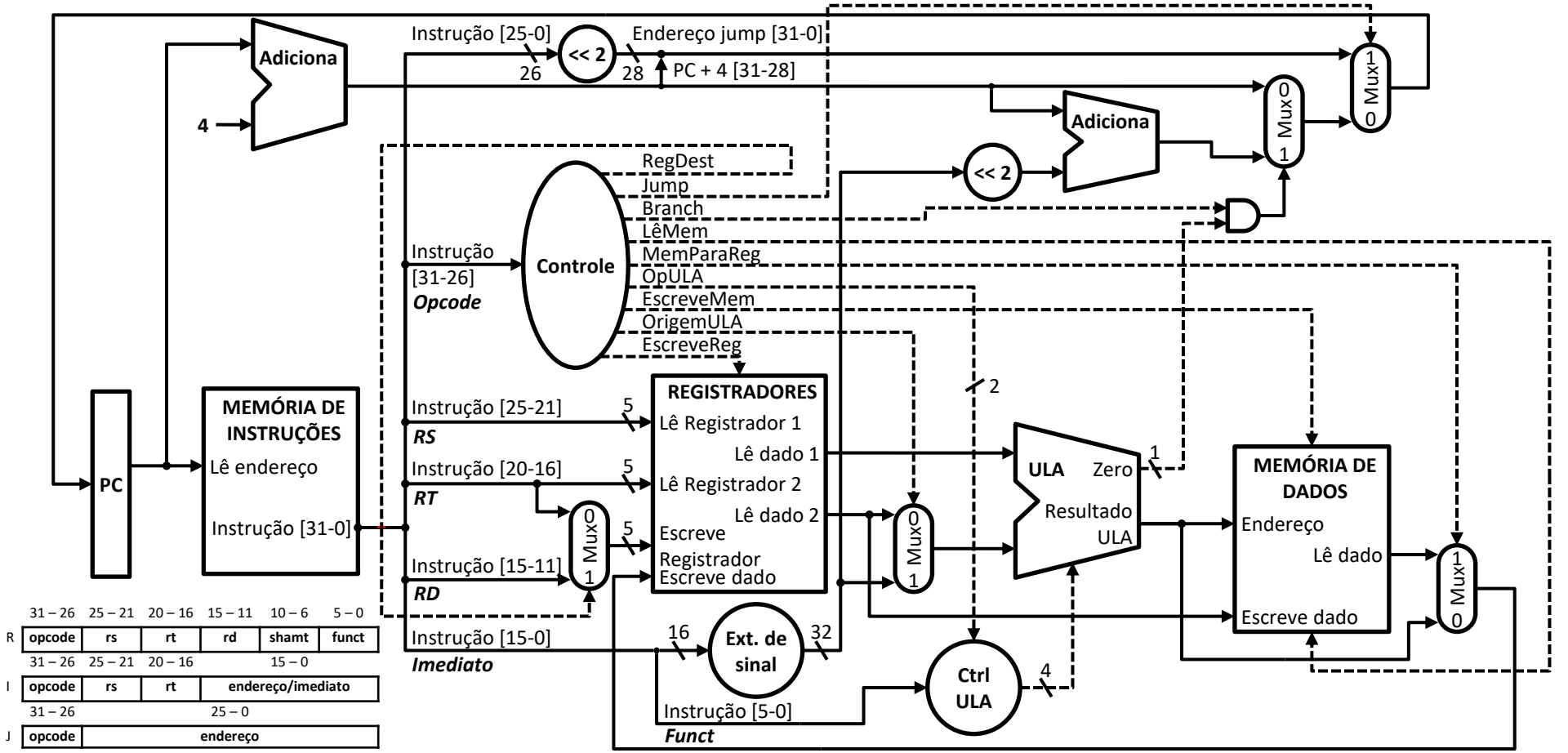
Roteiro

- MIPS Monociclo
- Acrescentando o controle
- O controle da ALU
- A unidade de controle principal
- Sinais da unidade de controle
- Instruções tipo R (operações lógicas e aritméticas)
  - add $t1, $t2, $t3
- Instrução LW
  - lw $t1, 16($t2)
- Instrução SW
  - sw $t1, 16($t2)
- Instrução BEQ
  - beq $t1, $t2, offset
- Instrução JUMP
  - j endereço
- Implementação de ciclo único

MIPS Monociclo

- Projeto de ciclo único:
  - Busca, decodificação e execução das instruções ocorre em um **único ciclo de clock**
  - Nenhum recurso do caminho de dados pode ser usados mais de uma vez por instrução
    - Aqueles que necessitam ser utilizados mais de uma vez **devem ser replicados**
    - EX: memória de instruções e de dados separados, mais de um somador.
  - Multiplexadores são necessários na entrada dos componentes compartilhados
    - Seleção da entrada correta para resolver conflitos
  - Sinais de escrita para controlar a gravação no banco de registradores e na memória de dados
- O tempo de ciclo é determinado pelo tamanho do caminho mais longo (com maior tempo de execução).

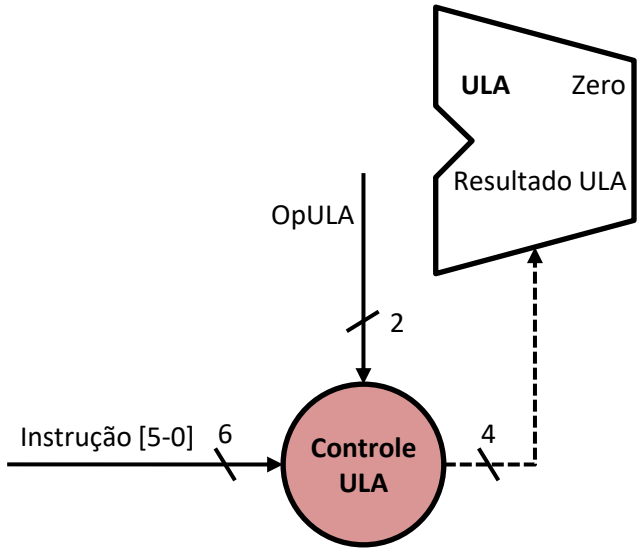
MIPS Monociclo



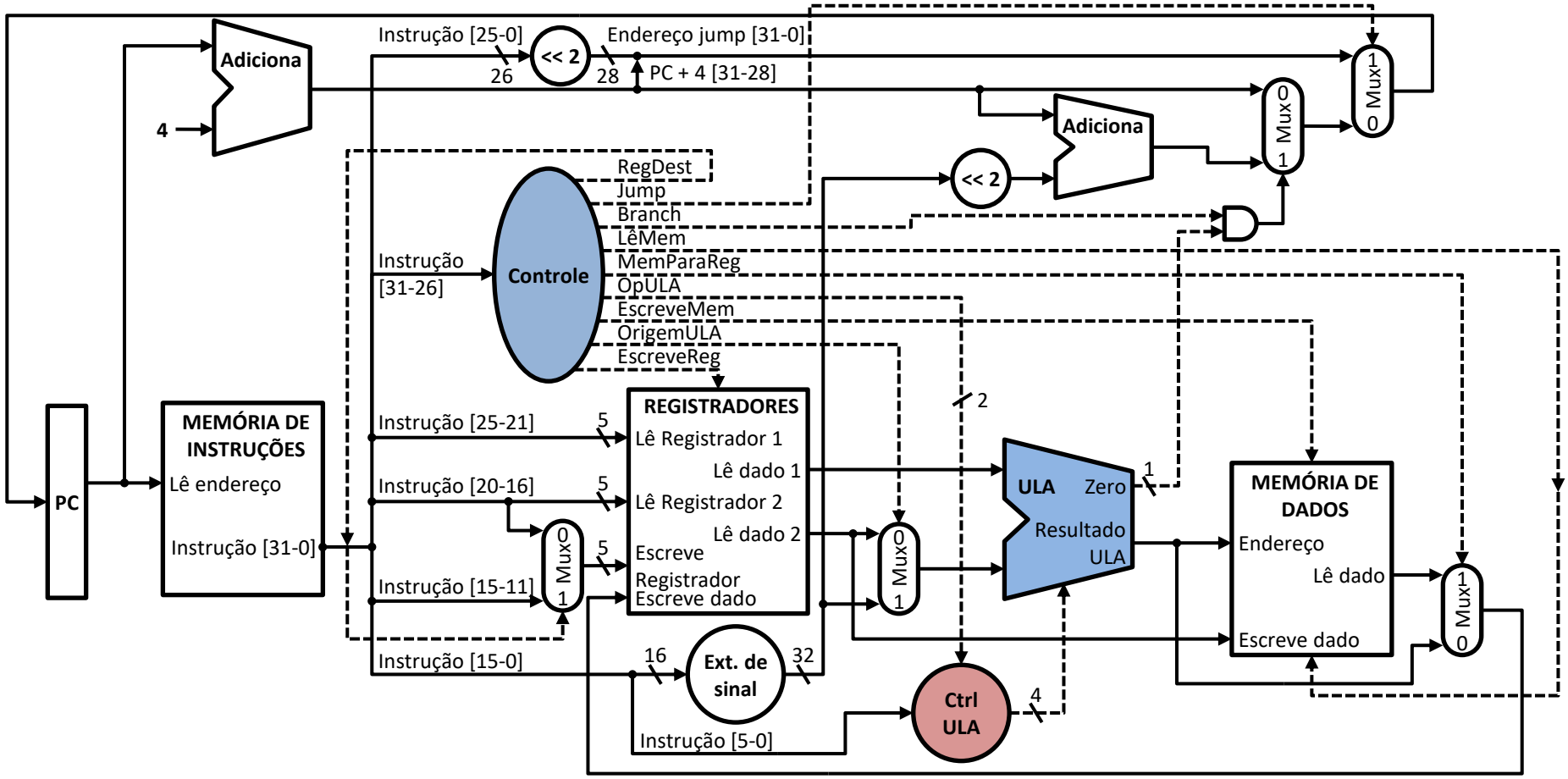
ACRESCENTANDO O CONTROLE

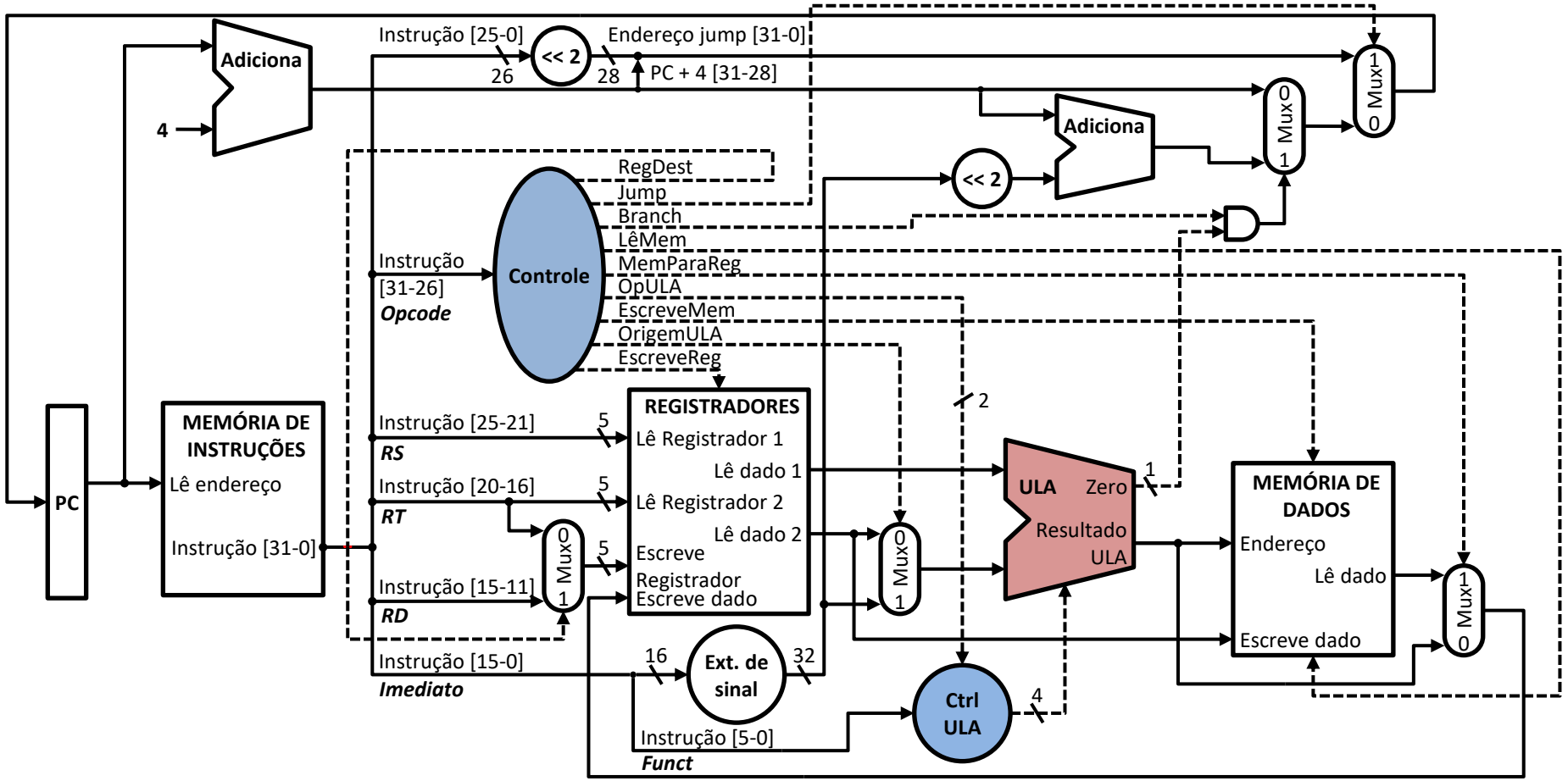
O controle da ALU

- A unidade de controle da ALU recebe duas entradas:
  - O campo funct da instrução
    - 6 bits
  - Um sinal de 2 bits (OpULA) vindo da unidade de controle principal OpALU.
    - 00: add para load e store
    - 01: sub para beq
    - 10: determinada pelo **campo funct**
- Gera um sinal de 4 bits para a ULA:
  - 0010: add
  - 0110: sub
  - 0000: and
  - 0001: or
  - 1100: nor
  - 0111: slt
  - ...



O controle da ALU




Adiciona
4
Instrução [25-0]
<< 2
26
28
Endereço jump [31-0]
PC + 4 [31-28]
Mux
0
1
Adiciona
<< 2
RegDest
Jump
Branch
LêMem
MemParaReg
OpULA
EscreveMem
OrigemULA
EscreveReg
Instrução [31-26]
Opcode
Controle
PC
MEMÓRIA DE INSTRUÇÕES
Lê endereço
Instrução [31-0]
Instrução [25-21]
RS
Instrução [20-16]
RT
Instrução [15-11]
RD
Instrução [15-0]
Imediato
REGISTRADORES
Lê Registrador 1
Lê dado 1
Lê Registrador 2
Lê dado 2
Escreve Registrador
Escreve dado
2
ULA
Zero
Resultado ULA
MEMÓRIA DE DADOS
Endereço
Lê dado
Escreve dado
Ext. de sinal
16
32
Ctrl ULA
4
Instrução [5-0]
Funct

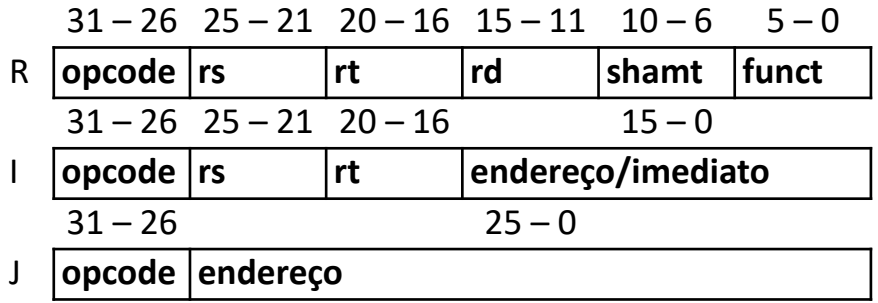
O controle da ALU

Opcode da instrução	OpALU	Operação da instrução	Campo funct	Ação da ALU desejada	Entrada do controle da ALU
LW	00	load word	XXXXXX	add	0010
SW	00	store word	XXXXXX	add	0010
BEQ	01	branch equal	XXXXXX	subtract	0110
tipo R	10	add	100000	add	0010
tipo R	10	subtract	100010	subtract	0110
tipo R	10	AND	100100	AND	0000
tipo R	10	OR	100101	OR	0001
tipo R	10	set on less than	101010	set on less than	0111

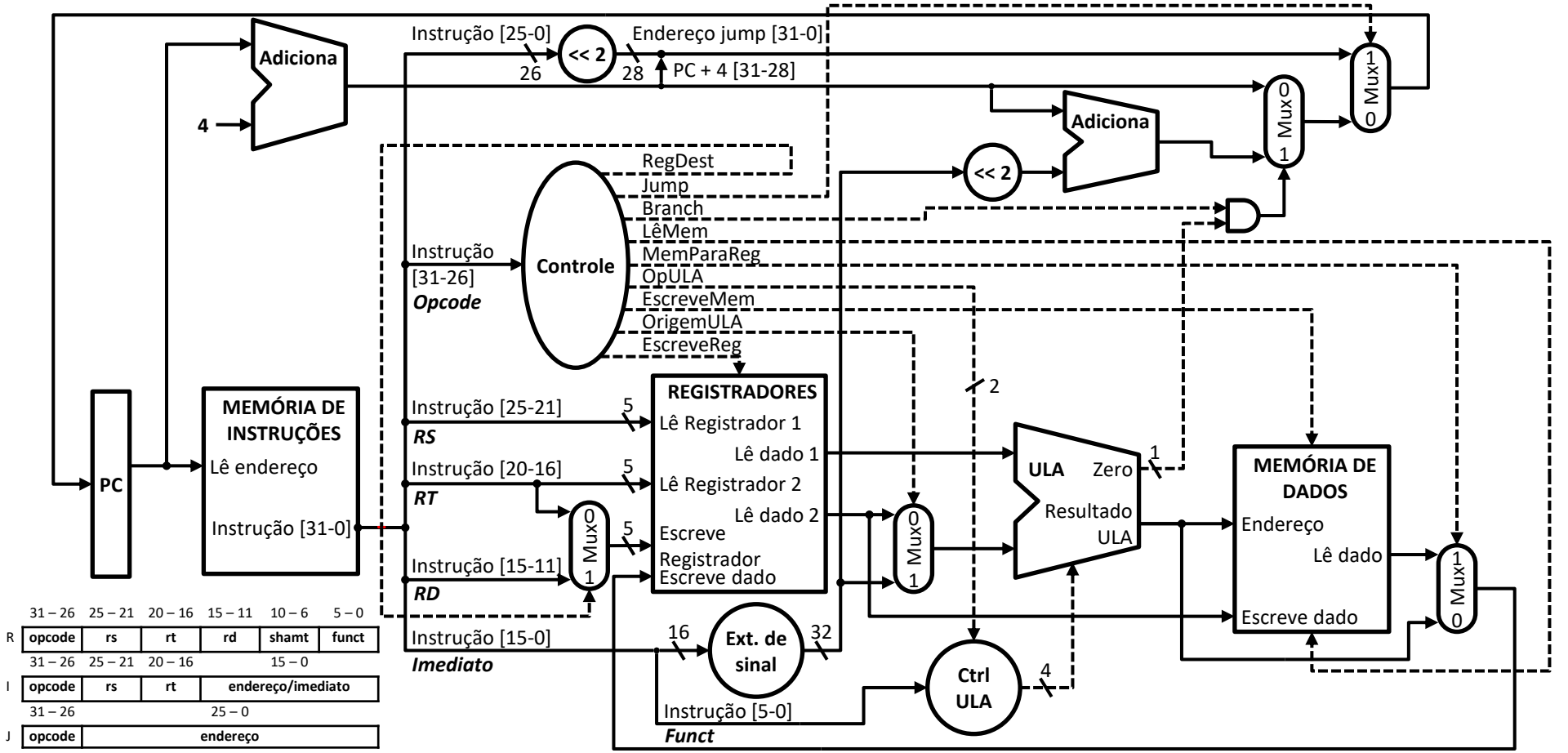
A UNIDADE DE CONTROLE PRINCIPAL

A unidade de controle principal

- Formatos de instrução considerados
  - Opcode: bits 31-26 (6 bits).
  - Registradores a serem lidos (rs e rt): bits 25-21 e 20-16 (5 bits cada)
    - Instruções tipo R, BEQ e store word
  - Registrador-base para instruções de load e store (rs): bits 25-21 (5 bits)
  - Constante de 16 bits: bits 15-0 (16 bits)
    - Deslocamento do BEQ e offset do lw e sw
  - Registrador destino:
    - Para lw (rt): bits 20-16 (5 bits).
    - Para tipo R (rd): bits 15-12 (5 bits)
    - Necessário um multiplexador antes do banco de registradores



A unidade de controle principal

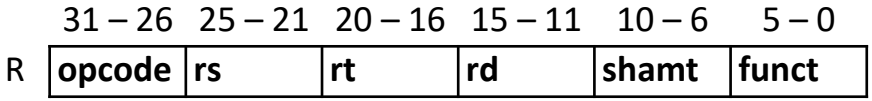


Sinais da unidade de controle

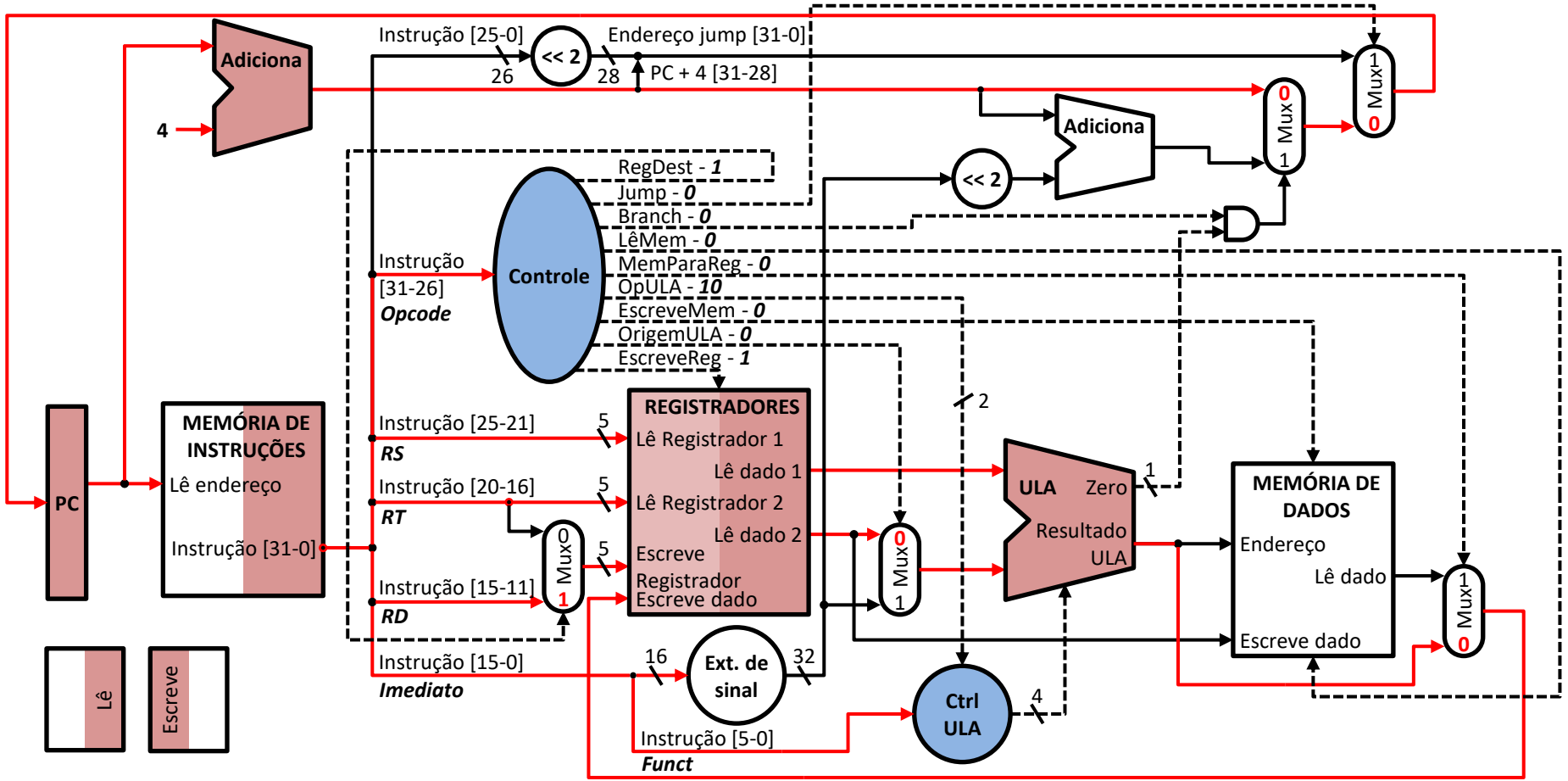
Nome do sinal	Efeito quando inativo	Efeito quando ativo
RegDst	O número do registrador destino para a entrada Registrador para escrita vem do campo rt (bits 20:16)	O número do registrador destino para a entrada ‘Registrador para escrita’ vem do campo rd (bits 15:11)
EscreveReg	Nenhum.	O registrador na entrada ‘Registrador para escrita’ é escrito como o valor da entrada ‘Dados para escrita’.
OrigALU	O segundo operando da ULA vem da segunda saída do banco de registradores (Dados de leitura 2).	O segundo operando da ULA consiste nos 16 bits mais baixos da instrução com sinal estendido.
OrigPC	O PC é substituído pela saída do somador que calcula o valor de PC + 4.	O PC é substituído pela saída do somador que calcula o destino do desvio.
LerMem	Nenhum.	O conteúdo da memória de dados designado pela entrada Endereço é colocado na saída ‘Dados da leitura’.
EscreveMem	Nenhum.	O conteúdo da memória de dados designado pela entrada Endereço é substituído pelo valor na entrada ‘Dados para escrita’.
MemParaReg	O valor enviado para a entrada ‘Dados para escrita’ do banco de registradores vem da ULA.	O valor enviado a entrada ‘Dados para escrita’ do banco de registradores vem da memória de dados.

Instruções tipo R (operações lógicas e aritméticas)

- **add $t1, $t2, $t3 # $t1 = $t2 + $t3 (rd = rs + rt)**
  1. Busca da instrução na memória de instruções e incremento do PC
  2. Dois registradores, rs e rt ($t2 e $t3), são lidos do banco de registradores
     A unidade de controle coloca valores nas linhas de controle
  3. A UAL opera sobre os dados lidos do banco de registradores, usando o campo funct (bits 5-0) para gerar a função da UAL
  4. O resultado da UAL é escrito no banco de registradores usando o campo rd (bits 15-11) da instrução para selecionar o registrador-destino ($t1)

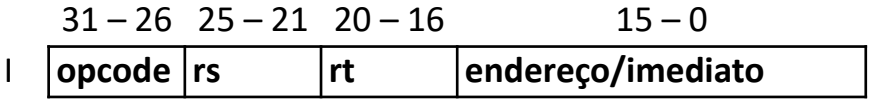


add $t1, $t2, $t3

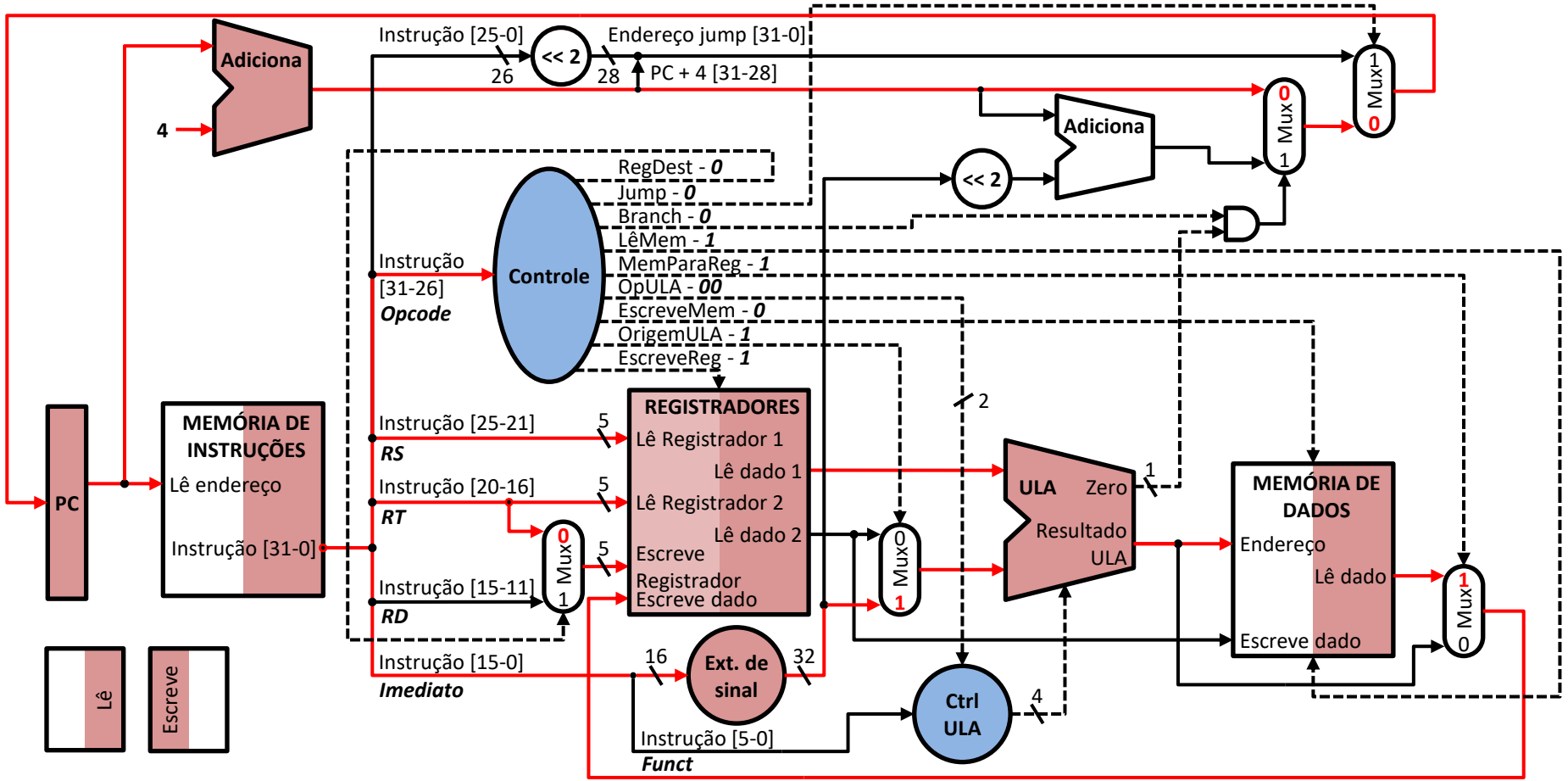

Adiciona
4
Instrução [25-0]
<< 2
26
28
Endereço jump [31-0]
PC + 4 [31-28]
Mux
0
1
Adiciona
<< 2
RegDest - 1
Jump - 0
Branch - 0
LêMem - 0
MemParaReg - 0
OpULA - 10
EscreveMem - 0
OrigemULA - 0
EscreveReg - 1
Instrução [31-26]
Opcode
Controle
PC
MEMÓRIA DE INSTRUÇÕES
Lê endereço
Instrução [31-0]
Instrução [25-21]
RS
Instrução [20-16]
RT
Instrução [15-11]
RD
Instrução [15-0]
Imediato
REGISTRADORES
Lê Registrador 1
Lê dado 1
Lê Registrador 2
Lê dado 2
Escreve Registrador
Escreve dado
5
2
ULA
Zero
Resultado ULA
1
MEMÓRIA DE DADOS
Endereço
Lê dado
Escreve dado
16
Ext. de sinal
32
Ctrl ULA
4
Instrução [5-0]
Funct
Lê
Escreve

Instrução LW

- **lw $t1, 16($t2)** **# lw RT, offset(rs) (rt = Memória(rs + offset))**
  1. Busca da instrução na memória de instruções e incremento do PC
  2. Leitura do conteúdo de um registrador (rs = $t2) do banco de registradores.
     A unidade de controle coloca valores nas linhas de controle
  3. Cálculo da soma do valor lido do banco de registradores (rs = $t2) com o resultado da extensão dos 16 bits menos significativos da instrução (offset = 16)
  4. O resultado da soma é usado para endereçar a memória de dados
  5. O dado vindo da unidade de memória é escrito no registrado indicado pelo campo (rt = $t1) (bits 20 – 16)

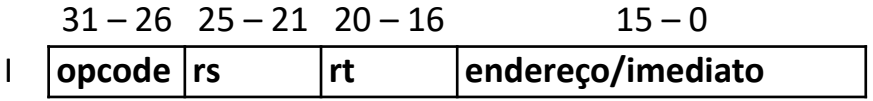


lw $t1, 16($t2)

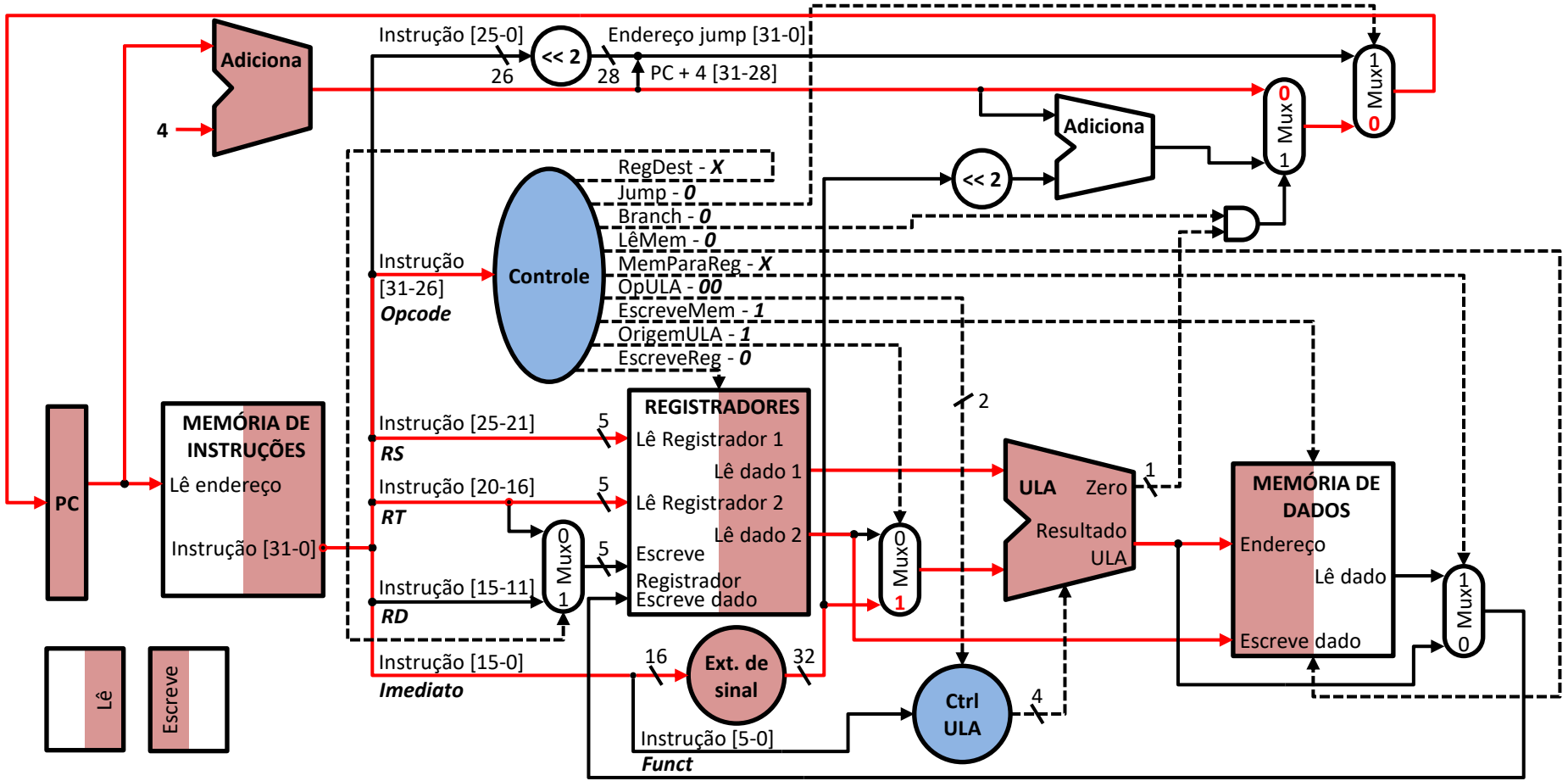

Adiciona
4
Instrução [25-0]
<< 2
26
28
Endereço jump [31-0]
PC + 4 [31-28]
Mux
1
0
Adiciona
<< 2
RegDest - 0
Jump - 0
Branch - 0
LêMem - 1
MemParaReg - 1
OpULA - 00
EscreveMem - 0
OrigemULA - 1
EscreveReg - 1
Instrução [31-26]
Opcode
Controle
PC
MEMÓRIA DE INSTRUÇÕES
Lê endereço
Instrução [31-0]
Instrução [25-21]
RS
Instrução [20-16]
RT
Instrução [15-11]
RD
Instrução [15-0]
Imediato
REGISTRADORES
Lê Registrador 1
Lê dado 1
Lê Registrador 2
Lê dado 2
Escreve Registrador
Escreve dado
5
2
ULA
Zero
Resultado ULA
MEMÓRIA DE DADOS
Endereço
Lê dado
Escreve dado
Ext. de sinal
16
32
Ctrl ULA
4
Instrução [5-0]
Funct
Lê
Escreve

Instrução SW

- **sw $t1, 16($t2) # sw rt, offset(rs) (Memória(rs + offset) = rt)**
  1. Busca da instrução na memória de instruções e incremento do PC
  2. Dois registradores, rs e rt ($t2 e $t1), são lidos do banco de registradores
     A unidade de controle coloca valores nas linhas de controle
  3. Cálculo da soma do valor lido do banco de registradores (rs = $t2) com o resultado da extensão dos 16 bits menos significativos da instrução (offset = 16)
  4. O valor lide de rt ($t1) é escrito na memória no endereço calculado em (3), rs + offset ($t2 + 16)



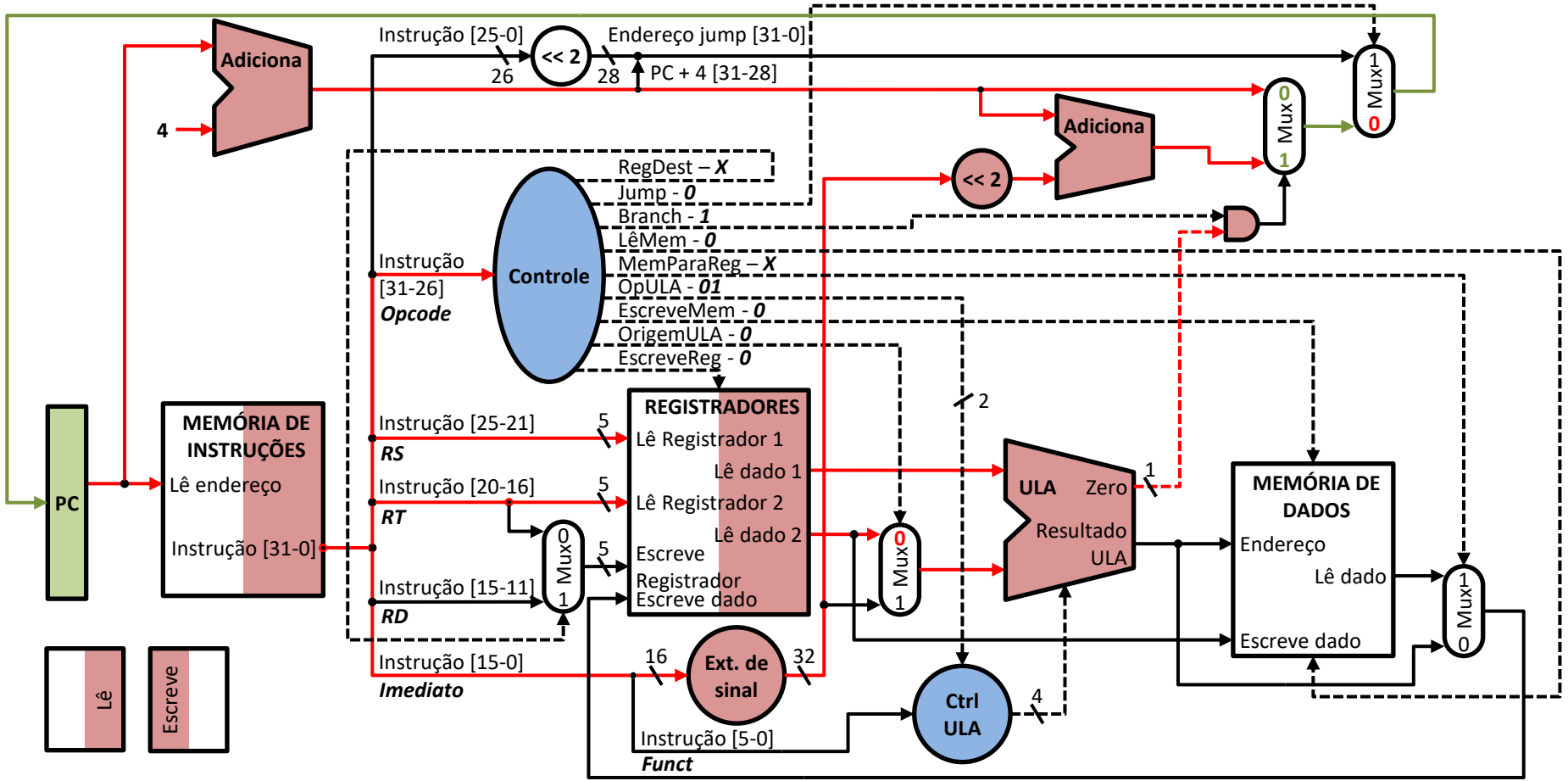
sw $t1, 16($t2)


Adiciona
4
Instrução [25-0]
<< 2
26
28
Endereço jump [31-0]
PC + 4 [31-28]
Mux
0
1
Adiciona
<< 2
RegDest - X
Jump - 0
Branch - 0
LêMem - 0
MemParaReg - X
OpULA - 00
EscreveMem - 1
OrigemULA - 1
EscreveReg - 0
Instrução [31-26]
Opcode
Controle
PC
MEMÓRIA DE INSTRUÇÕES
Lê endereço
Instrução [31-0]
Instrução [25-21]
RS
Instrução [20-16]
RT
Instrução [15-11]
RD
Instrução [15-0]
Imediato
REGISTRADORES
Lê Registrador 1
Lê dado 1
Lê Registrador 2
Lê dado 2
Escreve Registrador
Escreve dado
5
2
ULA
Zero
Resultado ULA
1
MEMÓRIA DE DADOS
Endereço
Lê dado
Escreve dado
16
Ext. de sinal
32
Instrução [5-0]
Funct
Ctrl ULA
4
Lê
Escreve

Instrução BEQ

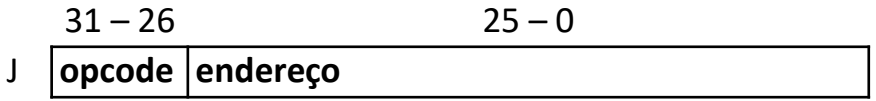
- **beq $t1, $t2, label # beq rs, rt, offset (se rs == rt, PC = PC + 4 + offset * 4)**
  1. Busca da instrução na memória de instruções e incremento do PC
  2. Dois registradores, rs e rt ($t2 e $t1), são lidos do banco de registradores
     A unidade de controle coloca valores nas linhas de controle.
  3. Realização de uma subtração pela ALU sobre os dois valores lidos do bando de registradores
     - O valor de PC+4 é somado ao resultado da extensão do sinal dos 16 bits menos significativos da instrução (deslocamento) deslocado de dois bits à esquerda
     - O resultado dessa soma é o endereço de destino do desvio
  4. A saída Zero da ALU é usada para decidir se o PC deve ser atualizado com o valor de PC+4 ou com o valor do endereço de destino do desvio condicional

beq $t1, $t2, offset

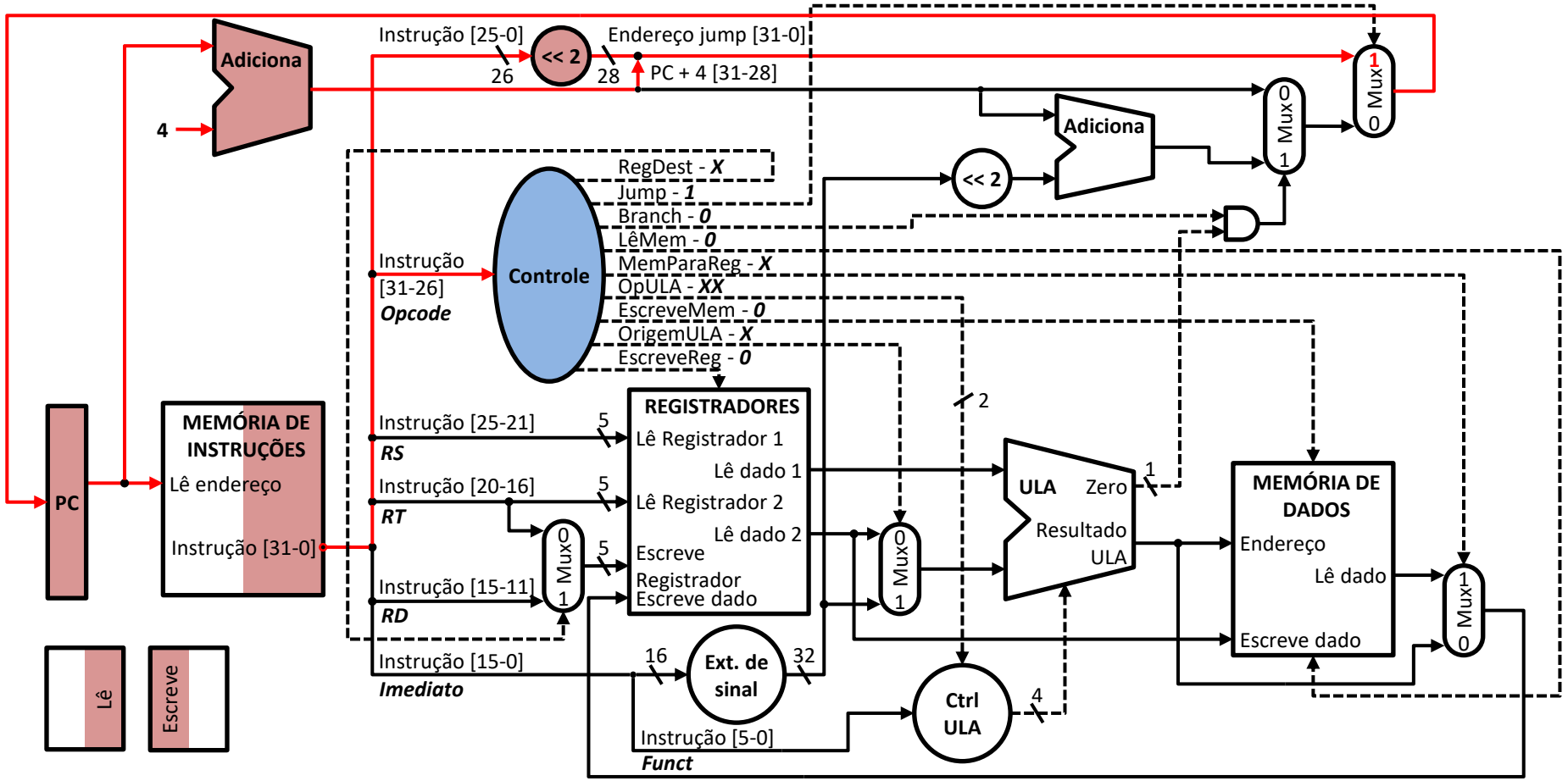


Instrução JUMP

- **J label # j endereço (PC = PC + 4 + endereço * 4)**
- A instrução jump se parece um pouco com uma instrução branch
  - Calcula o PC de destino de maneira diferente e não condicional
- Os 2 bits menos significativos são sempre 00.
  - Deslocamento de dois bits a esquerda (mult. por 4)
  - Os próximos 26 bits mais significativos vem do campo de 26 bits imediato da instrução
- Assim, pode-se implementar um jump armazenando no PC a concatenação dos:
  - 4 bits superiores do PC + 4 (bits 31:28)
  - O campo de 26 bits imediato da instrução jump
  - Os bits 00



j endereço


Adiciona
4
Instrução [25-0]
<< 2
26
28
Endereço jump [31-0]
PC + 4 [31-28]
Adiciona
<< 2
Mux
0
1
Mux
1
0
RegDest - X
Jump - 1
Branch - 0
LêMem - 0
MemParaReg - X
OpULA - XX
EscreveMem - 0
OrigemULA - X
EscreveReg - 0
Instrução [31-26]
Opcode
Controle
2
PC
MEMÓRIA DE INSTRUÇÕES
Lê endereço
Instrução [31-0]
Instrução [25-21]
RS
Instrução [20-16]
RT
Instrução [15-11]
RD
Instrução [15-0]
Imediato
Instrução [5-0]
Funct
5
5
5
REGISTRADORES
Lê Registrador 1
Lê dado 1
Lê Registrador 2
Lê dado 2
Escreve Registrador
Escreve dado
Mux
0
1
16
Ext. de sinal
32
Ctrl ULA
4
ULA
Zero
Resultado ULA
1
MEMÓRIA DE DADOS
Endereço
Lê dado
Escreve dado
Mux
1
0
Lê
Escreve

A unidade de controle principal

Instrução	RegDest	Jump	Branch	LêMem	MemParaReg	OpULA	EscreveMem	OrigemULA	EscreveReg
Lógicas e aritméticas add, sub, and, nor, slt, ...	1	0	0	0	0	10	0	0	1
Load word lw	0	0	0	1	1	00	0	1	1
Store word sw	X	0	0	0	X	00	1	1	0
Branchs beq, bne	X	0	1	0	X	01	0	0	0
Jump j	X	1	0	0	X	XX	0	X	0

Implementação de ciclo único

- O projeto de ciclo único é **ineficiente**
  - O clock precisa ter a mesma duração para cada instrução
    - O CPI será sempre 1
- O tempo de ciclo é determinado pelo **caminho mais longo**
  - Esse caminho corresponde a instrução **lw** que usa cinco unidades funcionais em série:
    - Memória de instruções
    - Banco de registradores
    - ALU
    - Memória de dados
    - Banco de registradores
  - Embora o CPI seja 1, o desempenho geral de uma implementação de ciclo único provavelmente não será bom
    - Outras instruções poderiam ser um ciclo de clock mais simples

Implementação de ciclo único

- Suponha os seguintes tempos de operação
  - Unidade de memória: 200 ps, ALU e somadores: 100 ps, Banco de registradores: 50 ps.
- E o mix de instruções:
  - 25% loads, 10% stores, 45% ALU, 15% desvios, 5% jumps

Tempo de execução = Contagem instruções × CPI × Tempo de ciclo

Tempo de execução = Contagem instruções × 1 × Tempo de ciclo

Tempo de execução = Contagem instruções × Tempo de ciclo

Classe de instrução	Memória de instrução	Banco de registradores	ULA	Memória de dados	Banco de registradores	Tempo total (ps)
Tipo R	200	50	100		50	400
Load word	200	50	100	200	50	600
Store word	200	50	100	200		550
Branch	200	50	100			350
Jump	200					200

Implementação de ciclo único

- Tempo exigido para cada classe de instruções:
  - *Considerando que os multiplexadores, unidade de controle, acessos ao PC, unidade de extensão do sinal e os fios não possuem atraso*

Classe de instrução	Busca de instrução	Leitura de registrador	Operação ULA	Leitura/escrita de dados	Escrita de registrador	Tempo total (ps)
Unidades funcionais usadas	Memória de instrução	Banco de registradores	ULA	Memória de dados	Banco de registradores	
Tipo R	200	50	100		50	400
Load word	200	50	100	200	50	600
Store word	200	50	100	200		550
Branch	200	50	100			350
Jump	200					200

- O tempo do ciclo de clock é determinado pela instrução mais longa:
  - Nesse caso, 600ps para a instrução Load Word (lw) ser executada

Implementação de ciclo único

- Ciclo único
  - Como precisamos considerar que o ciclo de clock é igual ao atraso de pior caso para todas instruções
    - Não é possível utilizar técnicas de implementação que reduzem o atraso do caso comum:
      - Violação dos princípios de projeto!
  - Na implementação de ciclo único **cada unidade funcional precisa ser duplicada**
    - Eleva o custo de implementação
    - O projeto de ciclo único é **ineficiente no desempenho e no custo**
  - Solução: utilizar uma **implementação multiciclo**
    - Dessa forma, as instruções podem utilizar mais de um ciclo de clock para serem executadas, dependendo de sua complexidade

Bibliografia

- PATTERSON, D. A.; HENNESSY, J. L. **Organização e Arquitetura de computadores: a interface hardware/software**. 3. ed. Rio de Janeiro: Elsevier, 2005.
  - CAPÍTULO 5
- Notas de aula da Profa. Mary Jane Irwin
  - CSE 331 Computer Organization and Design
  - http://www.cse.psu.edu/research/mdl/mji/mjicourses

FIM